

## Getting started with R

### Getting started

Install R and R-studio on your PC or Mac. This can be done from the following webpage:

**R:** <https://www.r-project.org/>

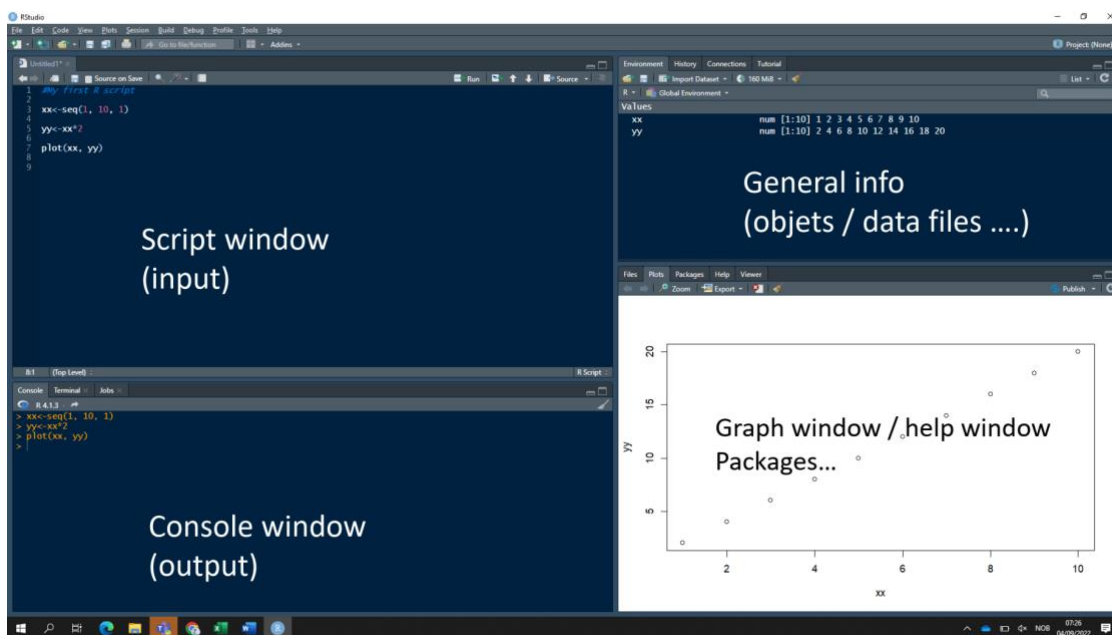
**R studio:** <https://www.rstudio.com/>

Once these two programs are installed, you only need to open R studio or any R file to start working. R studio will automatically start R.

### Becoming familiar with the R studio window.

Working with R means writing a script (upper left window) that contains the commands to R. This script contains your analysis (statistical model) as well as any notes you want to include to make it understandable.

When you run this script, the results appear in the “console” window (lower left). Graphs will appear in the “plots” window (lower right). Note that you if you ask for information about a specific command (e.g. if you want to know how to use the “mean” command you write: ?mean in the script window), the information will appear in the lower right window. In the upper right window, you find information of the objects you have created, data files you are working on and other background information.



To save the script, you simply need to use Ctrl (command on Mac) “s”, of the save option in the file menu. This will save only the script. Graphs can be saved separately using the “export” option.

### Working directory

The working directory is the folder in which you can save your document but also store the data files. It needs to be set at the start of the session. You can do it by the command:

`setwd("address of the folder on your PC")`, or using the menu: **Session / Set Working Directory / Choose Directory**

*In the following you will find small exercises to start working with R. If you already know how to use R, that is, if you know how to generate a script, save it, import a datafile and work with it, skip this first part. If not, go through this first part. The blue writings are script to help you in some of these exercises. Make sure you understand the meaning of the script.*

here are 2 ways to "run" a piece of code in R:

- Press the "Run" button at the top right of your script
- Press Ctrl + Enter

## Open R-studio and make the following calculations

$1+2 \times (3+4)$  {the multiplication sign in R is "\*"}

$4^3+3^2$  {power is written as for example:  $x^2$ }

$\sqrt{(4+3) \times (2+1)}$  {remember:  $\sqrt{a} = a^{\frac{1}{2}}$ }

$\sqrt{(3-4) \times (2+1)}$

$2\pi$  ( $\pi$  is written in R as "pi")

$\ln(12)$  (use the command log for natural log)

Compare the previous results with:  $\log(12, 10)$

## Create an object in R

With the following syntax:

```
x<-2
```

you create an object "x" with a value of 2.

You can check which object you have in your working file in R using

```
ls()
```

You can remove objects using the syntax:

```
rm(object name)
```

## Simple mathematical function in R

Considering  $x = 2.56$ , find the meaning of the different function

```
ceiling(x)
```

```
floor(x)
```

```
trunc(x)
```

```
round(x, y)
```

Here is a short list of simple mathematical functions defined in R

| Function | Meaning                         |
|----------|---------------------------------|
| Log(x)   | log to base $e$ of $x$          |
| exp(x)   | Antilog (or exponential) of $x$ |
| Log(x,n) | log to base $n$ of $x$          |

|                         |                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------|
| <code>sqrt(x)</code>    | Square root of $x$                                                                                 |
| <code>floor(x)</code>   | Greatest integer $< x$                                                                             |
| <code>ceiling(x)</code> | Next integer $> x$                                                                                 |
| <code>runif(n)</code>   | Generate $n$ random numbers between 0 and 1 from a uniform distribution, $n$ being the sample size |
| $x^n$                   | $x^n$                                                                                              |

---

### Create simple data set in R

You can generate small data set directly in R

Copy the following syntax to create a data set with 3 columns and 10 rows.

```
ind<-1:10
cat<-rep(1:2, each=5)
mass<-c(2, 2.3, 2.4, 1.4, 1.5, 2, 2.3, 4, 2.1, 3)
data1<-cbind(ind,cat,mass)
data1<-data.frame(data1)# this last command specify that
the names of the vector "ind", "cat" and "mass" are now the
name of the variables in a dataset.
```

For example, “rep” means repeat, “cbind” means combine the columns. If you are uncertain about the meaning of a command you can write: `?cbind`.

You have now a data set called “data1”. If you want to work with these data, you will have to specify it in the script. There is two ways to do this. For example, if you want to calculate the mean mass for the ten observations you can write either

```
>with(data1, mean(mass))
```

or

```
mean(data1$mass)
```

Similarly, to calculate the mean mass only for the category 2, you can write

```
with(data1, mean(mass[cat==2]))
```

or

```
mean(data1$mass[data1$cat==2])
```

### Importing and working on a data set

You can either use a line of script to read a datafile that is stored in your computer. For example the following script

```
seedling<-read.table("C:\\Rdatafile\\BI3051\\seedling.txt",
header=T)
```

will read the text file “seedling.txt” that is stored in the C drive of the computer in a folder called BI3051 itself placed in a folder called Rdatafile. Note that the datafile is now called “seedling”.

You can also use the menu “import Dataset” on the upper right window. Note that in all cases you will have to specify if the file is a text file, csv file or any other type of files. Once again, you can use the help function.

```
?read.table
```

```
?read.csv
```

### Built-in statistical function

R has a large number of pre-defined statistical functions. The most “common” are: `mean`, `sd`, `var`, `min`, `max`, `sum`, `summary`.

Use these functions on the data set “data1” that you created to calculate the mean, variance, standard deviation, ect of the “mass” variable.

You can also create your own function by writing

```
function(name of the function you want to create)
{...
}
```

When you have several categories, you can realise the previous operation for all categories simultaneously using the command `tapply`

```
tapply(mass,cat,mean)
```

Remember if you forgot what a function means, you can write

```
?mean
```

```
?var
```

to get the information.

### Create a vector

If you want to combine values (numerical or text) into a vector, the general command is

```
vect<-c(x1, x2, x3, ...) for numerical values
```

```
Vect<-c("x", "y", "z"... ) for text
```

### Subsetting and inspecting a vector

Run the following synthax and understand what it does:

```
animals <- c("orangutan","reindeer","koala","bee")
```

```
animals[2]
```

```
animals[c(2,3)]
```

```
animals[-c(3,2)]
```

```
weights <- c(21, 34, 39, 54, 55)
```

```
weights[c(TRUE, FALSE, TRUE, TRUE, FALSE)]
```

```
weights > 50
```

```
weights[weights > 50]
```

```
weights[weights <= 39]
```

```
weights[weights < 30 | weights > 50]
```

```
weights[weights <= 35 & weights != 21]
```

```
weights <= 35 & weights != 21
```

```
weights[weights > 35 & weights < 50]
```

For running simulations, it is often useful to generate a vector of random numbers extracted from different types of distribution (normal, binomial, gamma...). The syntax for number generated from a normal distribution is:

```
rnorm(number of individuals, mean, standard deviation)
```

You can do similar sampling from binomial distribution, poisson distribution or gamma distribution: rbinom, rpois, rgamma (see lecture on GLM).

### Create a matrix

```
A = matrix(  
  # Taking sequence of elements  
  c(1, 2, 3, 4, 5, 6, 7, 8, 9),  
  # No of rows  
  nrow = 3,  
  # No of columns  
  ncol = 3,  
  # By default matrices are in column-wise order  
  # So this parameter decides how to arrange the matrix  
  byrow = TRUE)  
# Naming rows  
rownames(A) = c("a", "b", "c")  
# Naming columns  
colnames(A) = c("c", "d", "e")  
cat("The 3x3 matrix:\n")  
print(A)
```

### An exercise to learn how to explore data, manage data sets, and calculate descriptive statistics and produce simple graphs (Dataset: seedling.txt)

The data correspond to the seed mass and the measurements of morphological and life history traits of seedlings at one month after emergence from different intra- and inter-population crosses of *Dalechampia scandens*.

The different columns represent:

*cross*: type of cross realised within between different populations

*hybrid*: hybrid (1) or non-hybrids (0)

*seedcat*: category of seeds. These categories are based on the size of the seeds in the maternal population.

*seedset*: total number of seeds contained in the fruit (max = 9)

*sdwght*: mass of one seed in g.

*Lobe*: size of the largest leaf of the seedling 1 month after germination

*biomass*: dry mass of the plant (including roots) one month after germination.

Open the data set in R and answer the following questions:

- What is the total number of observations and what is the number of observations per category (hybrid vs. non-hybrid)?
- What is the number of observations in each cross × hybrid category?
- Are the different variables normally distributed? (Visual analysis only)
- Explore graphically the possible correlation between variables.

- Represent the relationship between seed mass and plant biomass at one month for the two seed category on two graphs side by side, and then on a single graph where both categories appear with different symbols
- Calculate the mean and standard deviation of the seed mass for hybrid and non-hybrid crosses for the two seed categories, and represent the change in mean according to these two variables.

Useful script

```
data<-read.table("seedling.txt", header=T)
names(data)
length(data$variable name) this will give the total number of observation
table(variable 1, variable 2) This gives the number of observation for each
level for the factor 1 and 2 combined
par(mfrow=c(x,y)) allows to partition the graph window in x rows and y columns
hist(variable) provide the histogram for a given variable
plot(x,y, col=as.numeric(category), pch=as.numeric(category),
xlab=xxx, ylab=yyy) Provides a scatterplot with more than one category of points.
The label for xxx and yyy have to be defined in advance
tapply(variable1,list(category 1,category 2), mean, na.rm=T)
gives the mean for the variable 1 for the category 1 and 2 combined
interaction.plot(category 1, category 2, variable 1) graphical
representation of the mean of the variable 1 in the category 1 and 2 combined
```